

How Services Communicate

Part V continues — the protocols and architectures that connect a system's pieces.

- 26 RPC & gRPC** — calling a remote function, fast and typed
- 27 GraphQL** — ask for exactly the data you need
- 28 Message Queues & Async Communication** — decoupling in time
- 29 Microservices vs Monolith** — how to structure the whole system
- 30 Service Discovery & Service Mesh** — finding and connecting services

Chapters 26–28 cover the communication styles; 29–30 cover the architecture that uses them.

CHAPTER 26

RPC & gRPC

Part V · Communication & Architecture — calling a remote function, fast and typed.

LEARNING OBJECTIVES

- Explain what RPC is and how it differs from REST.
- Explain how gRPC uses HTTP/2 and Protocol Buffers for speed and typing.
- Describe gRPC's streaming modes.
- Choose gRPC vs REST for a given boundary.

REAL-WORLD ANALOGY — AN INTERCOM TO ANOTHER BUILDING

You press an intercom, ask a colleague in another building to do a task, and to you it feels just like asking someone across the desk. **RPC** makes a call to another machine *look* like a normal function call — the plumbing (packaging the request, sending it, unpacking the reply) is hidden. Just remember the fallacies of Chapter 19: it looks local, but a real, fallible network sits in the middle.

26.1 The problem

Inside a system, services call each other constantly (“east-west” traffic). REST works, but its text-based JSON over HTTP/1.1 is verbose and comparatively slow, and it can force over/under-fetching (Chapter 25). For high-volume internal calls we want something fast, strongly typed, and as easy as calling a local function — that is **RPC**.

26.2 What RPC is

A **Remote Procedure Call** invokes a function that runs on another machine. A generated **stub** on the client marshals (serializes) the arguments, sends them over the network, and a stub on the server unmarshals them, runs the function, and returns the result the same way.

the call looks local — but a real network hop happens underneath



26.3 gRPC

gRPC is a modern, widely-used RPC framework. It runs over **HTTP/2** (multiplexing, Chapter 4) and serializes with **Protocol Buffers**. You define the service **contract-first** in a `.proto` file, and gRPC generates client and server code in many languages — giving fast, strongly-typed, cross-language calls.

26.4 Protocol Buffers

Protocol Buffers (“protobuf”) is a schema-defined **binary** serialization format: smaller and faster to encode/decode than JSON, strongly typed, with safe **schema evolution** (add fields without breaking old clients). The trade-off versus JSON is that it isn’t human-readable and needs the schema to interpret.

26.5 Streaming

Because it uses HTTP/2, gRPC supports four call shapes: **unary** (one request, one response), **server-streaming** (one request, a stream back), **client-streaming** (a stream up, one response), and **bidirectional streaming**. This suits real-time feeds and large data far better than request/response REST.

26.6 gRPC vs REST

Use gRPC when...	Use REST when...
Internal service-to-service calls needing speed, typed contracts, streaming, and polyglot codegen.	Public/browser-facing APIs needing ubiquity, HTTP caching (Chapter 8), and human-readable payloads.

gRPC isn’t natively browser-friendly (it needs a gRPC-web proxy), which is one reason REST still rules public APIs.

Common mistakes

WATCH OUT

- Treating an RPC like a truly local call — no timeout, retry, or failure handling (Chapter 19).
- Using gRPC for a public browser API and fighting the tooling.
- Breaking a protobuf schema by reusing/removing field numbers.
- Choosing REST for a chatty internal hot path where gRPC would be far cheaper.

Interview questions

1. What is RPC, and how does it differ from REST?
2. How does gRPC achieve speed and strong typing?
3. What are Protocol Buffers’ advantages and trade-offs vs JSON?
4. Describe gRPC’s streaming modes.
5. When would you choose gRPC over REST?

Hands-on exercises

1. Write a `.proto` service with one unary and one server-streaming method.
2. Compare the byte size of a JSON payload and its protobuf equivalent.
3. Decide gRPC or REST for: internal recommendation service; public mobile API; live location stream.

4. Explain a safe way to add a field to a protobuf message.

Mini project

BUILD A GRPC SERVICE

Define a small `.proto` (e.g. a `UserService` with `GetUser` and a streaming `ListUsers`). Generate stubs, implement the server, and call it from a client. Then add timeouts and a retry, proving the call is remote, not local. Compare payload size and latency to an equivalent REST/JSON endpoint.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — INTERNAL PROTOCOLS

Decide your internal service-to-service protocol. Use **gRPC** for hot internal paths (feed service ↔ post service ↔ user service) with `.proto` contracts, and keep **REST** at the public edge (Chapter 25). Note where streaming helps (live notifications) and where you still need a gRPC-web bridge for the browser.

Chapter summary

- **RPC** makes a remote call look like a local function via generated stubs that marshal/unmarshal.
- **gRPC** uses **HTTP/2 + Protocol Buffers** for fast, typed, cross-language, contract-first calls.
- Protobuf is compact binary with safe schema evolution; not human-readable.
- gRPC supports **streaming**; it's ideal for internal traffic, while REST stays best for public/browser APIs.

CHAPTER 27

GraphQL

Part V · Communication & Architecture — ask for exactly the data you need.

LEARNING OBJECTIVES

- Explain the over/under-fetching problem GraphQL solves.
- Describe schema, queries, mutations, and resolvers.
- Explain GraphQL's benefits and its real costs (caching, N+1, complexity).
- Choose between GraphQL, REST, and gRPC.

REAL-WORLD ANALOGY — A PRECISE REQUEST SLIP

Instead of receiving a fixed bundle of documents (some you didn't want) or making five separate requests, you hand a librarian one slip listing *exactly* the chapters and figures you need, and get precisely that back in one trip. GraphQL lets the client write that slip.

27.1 The problem

REST endpoints return fixed shapes, so clients **over-fetch** (receive fields they don't need) or **under-fetch** (must call several endpoints to assemble a screen) — painful for mobile and for many different clients (Chapter 25). **GraphQL** lets the client specify exactly which fields and relationships it wants, in a single request.

REST — several calls

```
GET /users/42
```

```
GET /users/42/posts
```

```
GET .../posts/9/comments
```

3 round trips; each returns fixed fields

GraphQL — one query

```
query {
  user(id:42) {
    name
    posts { title }
  }
}
```

1 request → exactly these fields

27.2 Core concepts

GraphQL exposes a **single endpoint** backed by a strongly-typed **schema**. Clients send **queries** (reads) or **mutations** (writes) describing the exact shape they want; the server's **resolvers** (one function per field) fetch each piece. Because a query can traverse relationships, one request can gather data that would take many REST calls.

27.3 Benefits

No over- or under-fetching; a single round trip for complex, related data; a strongly-typed, self-documenting schema; and a client-driven API that serves many client types without custom endpoints (reducing the need for per-client BFFs, Chapter 9).

27.4 Costs and pitfalls

GRAPHQL ISN'T FREE

Caching is harder: a single POST endpoint can't use simple HTTP/CDN caching the way REST URLs can (Chapter 8). The **N+1 problem** reappears in resolvers — you need batching (a “dataloader”) to avoid a query per item (Chapter 12). Arbitrary client queries can be expensive or abusive, so you must add **depth/complexity limits**. And the server is simply more complex. For a small, fixed API, REST is often simpler.

27.5 GraphQL vs REST vs gRPC

Style	Best for
REST	Simple, cacheable, public APIs with fixed resources.
GraphQL	Flexible, client-driven reads across many relationships; many diverse clients.
gRPC	Fast, typed internal service-to-service calls and streaming.

Common mistakes

WATCH OUT

- Adopting GraphQL for a small fixed API that REST would serve more simply.
- Ignoring the resolver **N+1** problem (no batching/dataloader).
- No **query depth/complexity limits**, allowing expensive or malicious queries.
- Assuming HTTP caching works as it does for REST.

Interview questions

1. What problem does GraphQL solve versus REST?
2. Explain schema, query, mutation, and resolver.
3. Why is caching harder in GraphQL?
4. What is the resolver N+1 problem, and how do you fix it?
5. When would you pick GraphQL over REST or gRPC?

Hands-on exercises

1. Write a GraphQL query fetching a user with their posts and each post's comment count in one request.
2. Show the REST calls the same screen would need, and count the round trips.
3. Describe how a dataloader batches resolver calls.
4. Propose a query-complexity limit for a public GraphQL API.

Mini project

EXPOSE ONE SCREEN TWO WAYS

Take a screen that needs a user, their recent posts, and comment counts. Implement it with REST (count the round trips and wasted fields) and with a single GraphQL query. Add a dataloader to batch the comment-count resolver and confirm the N+1 queries collapse into one. Reflect on the caching you gave up.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — WHERE GRAPHQL FITS

Decide whether your mobile/web clients would benefit from a **GraphQL** layer for the feed and profile screens (many related fields, varied clients) while keeping REST/gRPC elsewhere. If you adopt it, specify resolver batching for feed items and a query-complexity limit to protect the backend.

Chapter summary

- **GraphQL** lets clients request **exactly** the fields and relationships they need in one round trip, ending over/under-fetching.
- It uses a typed **schema** with **queries/mutations** resolved by **resolvers**.
- Costs: harder **caching**, the resolver **N+1** problem (needs batching), and the need for **complexity limits**.
- Pick per boundary: REST (simple/cacheable), GraphQL (flexible reads), gRPC (fast internal).

CHAPTER 28

Message Queues & Async Communication

Part V · Communication & Architecture — decoupling services in time.

LEARNING OBJECTIVES

- Explain why synchronous calls couple services and when async is better.
- Explain how a message queue decouples producer and consumer.
- Distinguish point-to-point queues from publish-subscribe.
- Explain delivery guarantees, dead-letter queues, and backpressure.
- Compare Kafka and RabbitMQ at a high level.

REAL-WORLD ANALOGY — AN INBOX INSTEAD OF A PHONE CALL

To ask a busy colleague for something, you can phone them and wait on hold (synchronous), or drop a note in their inbox and carry on with your day (asynchronous). The inbox lets each of you work at your own pace, and if they're out, the note simply waits. A **message queue** is that inbox between services.

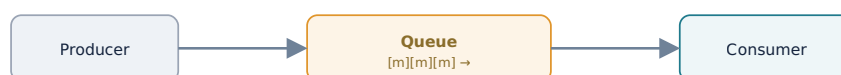
28.1 The problem

Synchronous calls (REST/gRPC) **couple** services: the caller waits for a reply, and if the callee is slow or down, the caller stalls or fails (Chapters 5, 19). But many tasks don't need an immediate answer — sending an email, processing an upload, updating a search index. **Asynchronous messaging** lets the caller hand off the work and move on.

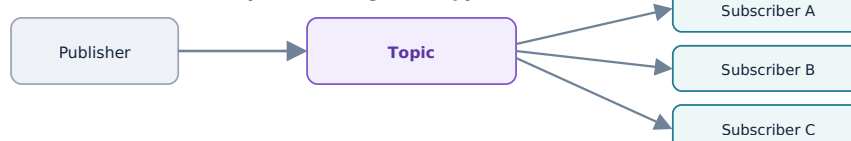
28.2 What a message queue is

A queue is a durable buffer between a **producer** and a **consumer**. The producer sends a message and returns immediately; the queue stores it; the consumer processes it when ready. Producer and consumer are **decoupled in time** (and don't need to know each other).

Point-to-point queue — one consumer handles each message



Publish-subscribe — every subscriber gets a copy



28.3 Benefits

Decoupling (services needn't be up at the same time), **load leveling** (the queue absorbs spikes so consumers work at a steady rate — recall the utilization curve, Chapter 5), **resilience** (messages wait if a consumer is down), **scalability** (add consumers to drain faster), and clean **offloading** of slow background work.

28.4 Two patterns

- **Point-to-point (queue)**: each message is handled by exactly **one** consumer — ideal for distributing work across a pool.
- **Publish-subscribe (topic)**: each message is delivered to **every** subscriber — ideal for broadcasting events (fan-out).

28.5 Delivery guarantees and hazards

KEY INSIGHT — AT-LEAST-ONCE MEANS DESIGN FOR DUPLICATES

Most queues offer **at-least-once** delivery: a message may be delivered more than once (e.g. if an ack is lost). So consumers must be **idempotent** (Chapter 25) — processing the same message twice must be safe. (At-most-once risks loss; exactly-once is costly and often approximated.) Also plan for **dead-letter queues** (park poison messages that keep failing), ordering needs, and **backpressure** when producers outrun consumers.

28.6 Kafka vs RabbitMQ

RabbitMQ	Kafka
A traditional broker with flexible routing and queues; great for task/work distribution and complex routing.	A distributed, partitioned, durable log ; extreme throughput and replayable event streams — ideal for event pipelines and analytics.

Common mistakes

WATCH OUT

- Making consumers that aren't idempotent under at-least-once delivery.
- No dead-letter queue, so one poison message blocks or loops forever.
- Ignoring backpressure and letting an unbounded queue grow without limit.
- Using async messaging where the caller genuinely needs an immediate answer.
- Assuming global ordering across partitions/consumers when it isn't guaranteed.

Interview questions

1. Why do synchronous calls couple services, and when is async better?
2. How does a message queue decouple producer and consumer?
3. Contrast point-to-point with publish-subscribe.
4. What does at-least-once delivery imply for consumers?

5. When would you choose Kafka vs RabbitMQ?

Hands-on exercises

1. Redesign a slow “send welcome email on signup” call as an async message.
2. Decide queue vs topic for: resizing uploaded images; broadcasting a “user updated” event.
3. Make a payment-processing consumer idempotent under duplicate delivery.
4. Describe when a message should go to a dead-letter queue.

Mini project

OFFLOAD WORK TO A QUEUE

Take a slow endpoint (e.g. image processing) and make it async: the API publishes a job to a queue and returns immediately; a worker consumes and processes it. Add an idempotency check so re-delivered messages don't double-process, and a dead-letter queue for repeated failures. Load-test and watch the queue absorb a spike while workers drain at a steady rate.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE ASYNC BACKBONE

Identify work to move off the request path: fan-out of a new post to followers' feeds, notifications, media processing, search indexing. Choose queue vs pub/sub for each, an event log (Kafka-style) for the activity stream, idempotent consumers, and dead-letter handling. This async backbone is what keeps your write path fast under load.

Chapter summary

- Async **message queues** decouple producer and consumer in time, absorbing spikes and surviving downstream outages.
- **Point-to-point** delivers each message to one consumer; **pub/sub** broadcasts to all subscribers.
- **At-least-once** delivery means consumers must be **idempotent**; plan dead-letter queues and backpressure.
- **RabbitMQ** excels at flexible routing/work; **Kafka** at high-throughput, replayable event streams.

CHAPTER 29

Microservices vs Monolith

Part V · Communication & Architecture — how to structure the whole system.

LEARNING OBJECTIVES

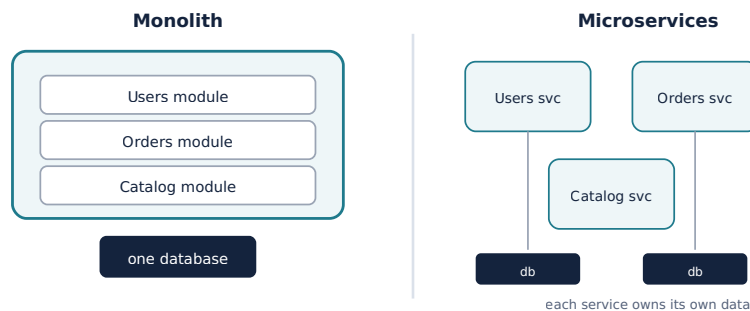
- Describe the monolith and microservices architectures and their trade-offs.
- Explain why microservices import all the distributed-systems complexity of Part IV.
- Explain why you should usually start with a monolith.
- Recognize the modular monolith and the distributed-monolith anti-pattern.

REAL-WORLD ANALOGY — ONE BIG BUILDING VS A CAMPUS

You can house an organization in one large building where everyone shares halls and utilities (a **monolith**) or across a campus of specialized buildings connected by paths (**microservices**). The single building is simpler to run and to walk around; the campus lets each team expand and renovate independently — but now people must travel between buildings, and the paths can get congested or wash out.

29.1 The problem

How should an application's code and deployment be structured? At one end sits the **monolith** — one codebase, one deployable. At the other, **microservices** — many small services, each independently deployable and owning its domain and data.



29.2 The monolith

Strengths

Simple to build, test, deploy, and debug; no network between modules (fast calls, easy ACID); ideal to start.

Weaknesses

Scales only as one unit; one bug or deploy can affect everything; hard for many teams to work in parallel; can rot into a “big ball of mud.”

29.3 Microservices

Strengths

Weaknesses

Independent scaling and deployment; team autonomy; technology diversity; fault isolation.

Imports **all of Part IV**: network calls, partial failure, cross-service data consistency (Sagas, Chapter 24), plus heavy ops and harder debugging (needs tracing, Chapter 50).

29.4 Which, and when

KEY INSIGHT — START MONOLITH; SPLIT WHEN THE PAIN JUSTIFIES IT

Microservices trade **code simplicity** for **organizational and scaling flexibility**, paid for in distributed-systems complexity. For most projects the right start is a **monolith** — ideally a **modular monolith** (clean internal module boundaries) so it can be split later. Move to microservices when team size, deployment friction, or independent-scaling needs genuinely demand it. Splitting too early yields the worst of both worlds: a **distributed monolith** — services so entangled they must be deployed together, with all the network cost and none of the independence.

Common mistakes

WATCH OUT

- Starting with microservices before the team or scale needs them.
- Splitting along the wrong boundaries, creating chatty, tightly-coupled services (a distributed monolith).
- Sharing one database across services, destroying their independence.
- Underestimating the ops, observability, and data-consistency burden.
- Letting a monolith rot instead of keeping clean module boundaries.

Interview questions

1. Compare monolith and microservices, with pros and cons.
2. Why do microservices bring distributed-systems complexity?
3. Why start with a monolith? What is a modular monolith?
4. What is a distributed monolith, and how do you avoid it?
5. Why should each microservice own its own data?

Hands-on exercises

1. List three signals that a monolith should be split into services.
2. Draw service boundaries for an e-commerce app and justify them.
3. Identify a cross-service operation that now needs a Saga.
4. Explain why sharing a database across services is problematic.

Mini project

EXTRACT ONE SERVICE

Start from a small modular monolith with two or three modules. Extract **one** module (say “notifications”) into a separate service communicating over the network. Feel the new costs directly: a network hop, failure handling, separate deployment, and no shared transaction. Decide whether the split was worth it for that module.

| Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE ARCHITECTURE DECISION

Decide your platform’s structure. Start as a **modular monolith** with clean boundaries (users, posts, feed, notifications). Then justify which one or two modules earn extraction into services first (e.g. feed generation for independent scaling), each with its own data, and specify the Saga for any operation that spans them.

| Chapter summary

- A **monolith** is one deployable — simple and fast to start, but scales and evolves as one unit.
- **Microservices** are independently deployable, domain-owning services — flexible, but they import all of Part IV’s distributed complexity.
- Usually **start with a monolith** (ideally modular) and split when team/scale/deploy pain justifies it.
- Avoid the **distributed monolith**; give each service its own data.

CHAPTER 30

Service Discovery & Service Mesh

Part V · Communication & Architecture — finding and connecting services at scale.

LEARNING OBJECTIVES

- Explain why service discovery is needed when instances come and go.
- Compare client-side and server-side discovery.
- Explain the east-west traffic problem a service mesh solves.
- Describe the sidecar model and what the mesh handles.
- Weigh the mesh's costs.

REAL-WORLD ANALOGY — A LIVE DIRECTORY AND A SWITCHBOARD

In a big campus, people constantly join, leave, and move offices, so you keep a **live directory** that's always current — that's **service discovery**. And rather than every caller learning how to place secure, retried, load-balanced calls, a dedicated **switchboard** next to each office handles all of that — that's the **service mesh** sidecar.

30.1 The problem

In a microservices system, instances constantly appear and vanish — autoscaling (Chapter 10), failures and restarts (Chapter 19) — so their addresses keep changing. Hard-coding hosts breaks (recall the fallacy “topology doesn't change”). So how does service A find a *healthy* instance of service B?

30.2 Service discovery

A **service registry** tracks which instances are currently healthy. Services **register** on startup and **deregister** (or fail health checks) on shutdown; callers look up live instances.

- **Client-side discovery:** the caller queries the registry and picks an instance itself.
- **Server-side discovery:** the caller hits a load balancer/router that consults the registry.

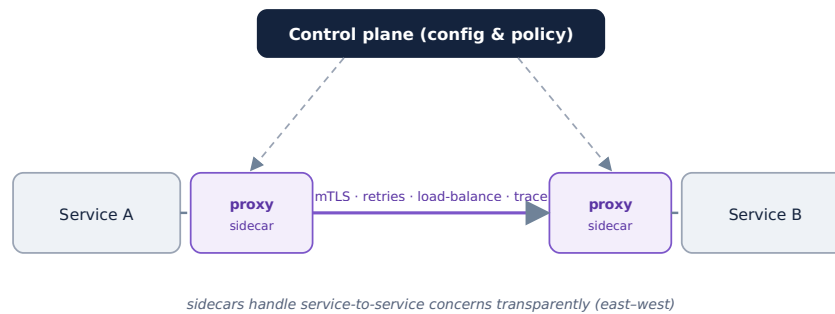
Common tools: Consul, etcd, Eureka, and Kubernetes' built-in DNS-based discovery.

30.3 The east-west problem

Once many services call each other, they *all* need the same cross-cutting concerns on every call: retries, timeouts, mutual TLS, load balancing, circuit breaking, and tracing (Chapters 9, 19). Re-implementing that in every service, in every language, is duplicated and error-prone.

30.4 The service mesh

A **service mesh** moves that logic *out* of application code into a **sidecar proxy** deployed next to each service instance. All traffic flows through the sidecars, which handle it transparently. The sidecars form the **data plane**; a **control plane** configures them centrally.



KEY INSIGHT — GATEWAY IS THE EDGE, MESH IS THE INTERIOR

The **API gateway** (Chapter 9) governs **north-south** traffic entering the system; the **service mesh** governs **east-west** traffic *between* internal services. Together they cover all the cross-cutting concerns without burdening application code — the gateway at the front door, the mesh in the hallways.

30.5 The costs

A MESH ISN'T FREE

Every call now passes through two extra proxies, adding **latency** and **resource cost**, and the mesh itself is another complex system to operate. It pays off when you have **many** services and genuinely need uniform security, resilience, and observability. For a handful of services, it's overkill — discovery plus a library may be enough.

Common mistakes

WATCH OUT

- Hard-coding service addresses instead of using discovery.
- Adding a full service mesh for only a few services (overkill).
- Confusing the mesh (east-west, internal) with the API gateway (north-south, edge).
- Ignoring the extra latency and resource overhead the sidecars add.

Interview questions

1. Why is service discovery necessary in a microservices system?
2. Compare client-side and server-side discovery.
3. What east-west problem does a service mesh solve?
4. What does a sidecar proxy handle, and what are data plane vs control plane?
5. When is a service mesh not worth it?

Hands-on exercises

1. Explain what happens to a hard-coded caller when its target autoscales to new IPs.
2. List four concerns a sidecar handles that you'd otherwise code in every service.

3. Decide gateway or mesh for: authenticating an incoming user; encrypting service A↔B calls.
4. Argue for or against a mesh for a 4-service system.

Mini project

DISCOVERY, THEN A SIDECAR

Run two services with a registry (e.g. Consul or Kubernetes DNS) so one finds healthy instances of the other without hard-coded addresses; scale the target up and down and watch calls follow. Then place a sidecar proxy in front of each and let it handle retries and mTLS, removing that logic from your application code.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — CONNECTING THE SERVICES

For the services you extracted in Chapter 29, add **service discovery** so they find each other as instances scale. Decide whether the number of services justifies a **service mesh** for mTLS, retries, and tracing, or whether discovery plus a shared client library suffices for now. Draw the boundary: gateway at the edge (Chapter 9), mesh (if any) between services.

Chapter summary

- **Service discovery** lets services find healthy instances via a registry as instances come and go — client-side or server-side.
- Many inter-service calls need the same cross-cutting concerns; a **service mesh** moves them into **sidecar proxies**.
- Sidecars (data plane) handle mTLS, retries, load balancing, and tracing; a **control plane** configures them.
- The **gateway** governs edge (north-south) traffic; the **mesh** governs internal (east-west) traffic — use a mesh only when you have enough services to justify it.

COMING NEXT — PART VI

Chapter 34 onward — Reliability & Resilience. With communication and architecture in place, Part VI makes systems survive failure: redundancy, timeouts and retries, circuit breakers, bulkheads, rate limiting, graceful degradation, and the Saga pattern for cross-service consistency.